

Solidity 타입 시스템과 함수 구조

값 타입 · 참조 타입 · 저장 위치 · 가시성 · modifier · 이벤트
강의 40min + 실습 20min

타입 시스템

함수 구조

modifier

01

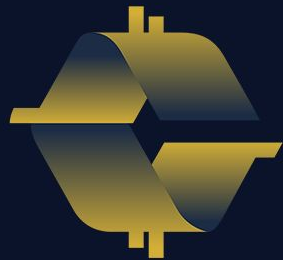
타입 시스템 값 타입

정수형 · 주소형 · 불리언 · enum · struct

uint / int

address

enum · struct



COINCRAFT
ACADEMY

타입 시스템 — 값 타입 (1/3)

SOLIDITY VALUE TYPES — INTEGER

원칙 — Solidity는 정적 타입 언어다. 변수 선언 시 타입을 반드시 명시해야 한다. 컴파일 타임에 타입 불일치를 잡아낸다.

타입	범위	크기
<code>uint8</code>	0 ~ 255	8bit
<code>uint128</code>	0 ~ $2^{128}-1$	128bit
<code>uint256</code>	0 ~ $2^{256}-1$	256bit (기본)
<code>int8</code>	-128 ~ 127	8bit 부호 있음
<code>int256</code>	$-2^{255} \sim 2^{255}-1$	256bit (기본)

0.8.0 이후 오버플로우 자동 revert — SafeMath 불필요. 의도적 wrap은 `unchecked{ }` 블록 사용.

오버플로우 처리

```
uint8 x = 255;
x += 1; // ❌ revert (0.8.0+)

// 의도적 wrap (gas 절약)
unchecked {
    x += 1; // → 0 (wrap)
}

// uint vs int 기본 타입
uint // = uint256
int  // = int256
```

실무 팁 — 토큰 잔액·수량엔 `uint256`. `uint8` 은 enum 인덱스·작은 카운터에만.

타입 시스템 — 값 타입 (2/3)

SOLIDITY VALUE TYPES — ADDRESS & BOOL

address — 20바이트 이더리움 주소

- `address` — 주소 저장. ETH 전송 불가.
- `address payable` — ETH 전송 가능 (`transfer`, `send`, `call`)
- `msg.sender`의 타입은 `address`
- ETH 전송 시 명시적 캐스팅 필요: `payable(addr)`
- `address(this)` — 현재 컨트랙트 주소
- `address(this).balance` — 보유 ETH (wei)

```
address owner = msg.sender;
payable(owner).transfer(1 ether);
// address → address payable 캐스팅
```

bool — 참/거짓

- `true` / `false` 두 값만 존재
- 연산자: `&&`, `||`, `!`, `==`, `!=`
- 단축 평가 — `a && b`에서 `a`가 `false`면 `b` 미실행
- gas 최적화 — 비용 큰 조건을 오른쪽에

```
bool paused = false;

// 비용 큰 조건은 오른쪽에
require(
    !paused && isEligible(msg.sender),
    "blocked"
);
// paused=true이면 isEligible 호출 안 함
```

단축 평가 — 왼쪽 조건이 결과를 확정하면 오른쪽은 실행하지 않는다. gas 절약 포인트.

타입 시스템 — 값 타입 (3/3)

SOLIDITY VALUE TYPES — ENUM & STRUCT

enum — 이름 있는 정수 집합

```
enum Status { Pending, Active, Closed }  
// 내부: uint8 (0, 1, 2)  
  
Status public state = Status.Pending;  
  
function activate() external {  
    require(state == Status.Pending, "Not pending");  
    state = Status.Active;  
}
```

- 내부는 `uint8` — 0부터 순서대로
- 상태 머신 패턴에 필수
- 범위 밖 값 대입 → 컴파일 에러

struct — 사용자 정의 복합 타입

```
struct Voter {  
    address addr;  
    uint256 weight;  
    bool    voted;  
}  
  
mapping(address => Voter) public voters;  
  
function register(address v) external {  
    voters[v] = Voter(v, 1, false);  
}
```

- storage / memory 모두 가능
- struct 안 mapping → storage 전용
- 패킹 규칙 적용 — 선언 순서 중요

설계 팁 — 자주 함께 읽히는 필드는 struct로 묶어라. 별도 mapping 3개보다 `mapping(address => Voter)` 1개가 SLOAD 절약.

02

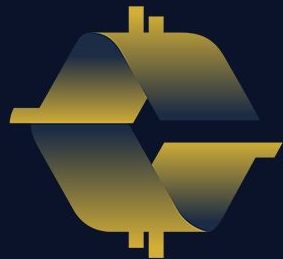
타입 시스템 참조 타입과 저장 위치

storage · memory · calldata · mapping · 패킹

storage

memory

calldata



COINCRAFT
ACADEMY

타입 시스템 — 참조 타입과 저장 위치

REFERENCE TYPES — DATA LOCATION (STORAGE / MEMORY / CALldata)

필수 규칙 — 참조 타입(`array` , `mapping` , `struct` , `bytes` , `string`)은 선언 시 저장 위치를 반드시 명시해야 한다. 값 타입은 불필요.

키워드	위치	특성	gas 비용
<code>storage</code>	블록체인 영구 저장소	상태변수 기본값. 트랜잭션 종료 후에도 유지	★★★ 비쌈
<code>memory</code>	함수 실행 중 임시	함수 종료 시 소멸. 수정 가능	★★ 중간
<code>calldata</code>	외부 함수 인자 전용	읽기 전용. 복사 없이 직접 참조 → 최저비용	★ 저렴

calldata 권장 — 외부 함수의 배열-struct 인자는 `calldata` 사용.
memory보다 저렴하고 안전 (수정 불가로 실수 방지).

함정 — 로컬 변수에 `storage` 포인터 사용 시 초기화 필수. 미초기화 → 슬롯 0 가리킴.

참조 타입 — storage 포인터 흔한 함정

STORAGE POINTER — COMMON PITFALL

❌ 위험 — 초기화 없는 포인터

```
struct Item { uint256 val; }
Item[] public items;

function buggy() external {
    Item storage ptr; // 슬롯 0 가리킴!
    ptr.val = 100;    // items[0].val 수정
                    // ← 의도하지 않은 변경
}
```

Solidity 0.6.0+ — 초기화 없는 로컬 storage 포인터는 컴파일 에러로 변경됨. 구버전 코드 감사 시 주의.

✅ 올바른 — 명시적 참조

```
function update(uint256 idx) external {
    // 기존 항목 명시적 참조
    Item storage ptr = items[idx];
    ptr.val = 100; // items[idx].val 수정
}

function readOnly(uint256 idx) external view {
    // 수정 없이 읽기만 → memory 복사
    Item memory copy = items[idx];
    // copy 수정은 storage에 영향 없음
}
```

규칙 — storage 포인터는 항상 기존 상태변수 항목을 대입해서 초기화. 읽기 전용이면 memory 복사.

참조 타입 — mapping의 특성

MAPPING — HASH-BASED KEY-VALUE STORE

특성	내용
선언	<code>mapping(KeyType => ValueType)</code>
저장 위치	storage 전용 (memory 사용 불가)
기본값	존재하지 않는 키 → 값 타입의 zero 반환
순회	불가 — 키 목록이 없음
중첩	<code>mapping(address => mapping(address => uint))</code>
삭제	<code>delete map[key]</code> → zero 값으로 초기화

내부 구조 — 실제 데이터는 `keccak256(key, slot)` 위치에 저장. "키가 없다"가 아니라 "값이 기본값"이다.

패턴 — 순회 필요 시 키 배열 별도 관리

```
// ERC-20 패턴
mapping(address => uint256) public balances;

// allowance 중첩 mapping
mapping(address => mapping(address => uint256))
    public allowances;

// 순회용 키 배열 (별도 관리)
address[] private _holders;

function _addHolder(address acc) internal {
    if (balances[acc] == 0)
        _holders.push(acc);
}
```

주의 — 홀더 배열이 무한 증가하면 gas limit 초과 위험. 순회 범위를 항상 제한하라.

참조 타입 — storage 패킹

STORAGE PACKING — DECLARATION ORDER DETERMINES GAS COST

❌ 패킹 실패 — 슬롯 4개

```
contract Unpacked {  
    uint64 a; // slot 0 (8B 혼자)  
    uint256 b; // slot 1 (32B)  
    uint64 c; // slot 2 (8B 혼자)  
    uint128 d; // slot 3 (16B 혼자)  
}  
// SSTORE/SLOAD: 4회
```

✅ 패킹 성공 — 슬롯 2개

```
contract Packed {  
    uint64 a; // slot 0 (8B)  
    uint64 c; // slot 0 (8B) ← 함께!  
    uint128 d; // slot 0 (16B) ← 꼭참!  
    uint256 b; // slot 1 (32B)  
}  
// SSTORE/SLOAD: 2회 → 절반
```

슬롯 1개 = 32바이트 — 같은 슬롯에 여러 작은 변수를 묶으면 SSTORE/SLOAD 횟수가 줄어 gas 절약. SLOAD cold = 2,100 gas.

패킹 규칙 — 작은 타입끼리 연속 선언. 32바이트 초과 시 다음 슬롯으로 넘어감. `uint256` 은 항상 단독 슬롯.

자주 같이 읽히는 변수 — 같은 슬롯에 배치하면 한 번의 SLOAD로 모두 읽을 수 있다. 접근 패턴 분석 후 순서 결정.

03

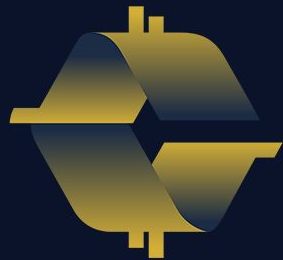
상태변수 가시성

public · private · internal — 노출 범위 최소화 원칙

public

private

internal



COINCRAFT
ACADEMY

상태변수 가시성

STATE VARIABLE VISIBILITY — PUBLIC / PRIVATE / INTERNAL

가시성	접근 가능 범위	getter 자동 생성	사용 예
public	내부 + 외부 + 상속 컨트랙트	✅ 자동 생성	토큰 잔액, 오너 주소
private	해당 컨트랙트만	❌ 없음	내부 상태, 비공개 변수
internal	해당 컨트랙트 + 상속 컨트랙트	❌ 없음	상속 기반 설계 변수
external	상태변수에는 사용 불가 — 함수 전용		

```
// public → getter 자동 생성
uint256 public totalSupply;
// 외부에서 totalSupply() 호출 가능

// private → 다른 컨트랙트 접근 불가
address private _secretKey;
```

오해 주의 — `private` 이어도 블록체인 데이터는 공개다. "다른 컨트랙트 읽기 금지"이지 "체인에서 숨기기"가 아니다.

기본값 — 가시성 미지정 시 `internal` 이 기본. 명시 권장.

04

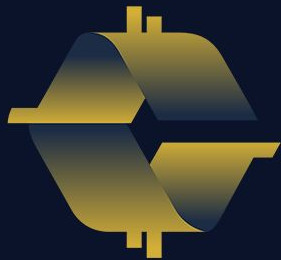
함수 구조 가시성·상태 변경성

가시성 4종 · view / pure / payable · 에러 처리

external / public

view · pure

require · revert



COINCRAFT
ACADEMY

함수 구조 — 가시성·상태 변경성 (1/3)

FUNCTION STRUCTURE — VISIBILITY (4 TYPES)

function 함수명 (인자) **가시성** 상태변경성 **returns** (반환타입) { ... }

가시성	내부 호출	외부 호출	상속 컨트랙트	특징
public	✓	✓	✓	가장 개방적. ABI에 포함
private	✓	✗	✗	해당 컨트랙트 내부만
internal	✓	✗	✓	상속 시 접근 가능
external	this.f() 만	✓	this.f() 만	calldata 직접 참조 → gas 절약

external 권장 — 외부에서만 호출되는 함수는 **external** 로. 배열 인자를 calldata로 받아 복사 없이 처리 → gas 절약.

실수 주의 — 모든 함수를 **public** 으로 선언하는 습관은 공격 표면을 넓힌다. 최소 노출 원칙.

함수 구조 — 가시성·상태 변경성 (2/3)

FUNCTION STRUCTURE — STATE MUTABILITY (VIEW / PURE / PAYABLE)

키워드	상태 읽기	상태 쓰기	ETH 수신
(없음)	✓	✓	✗
payable	✓	✓	✓
view	✓	✗	✗
pure	✗	✗	✗

외부 호출 시 gas — `view` / `pure` 는 로컬 실행 시 gas 0. 컨트랙트 내부 호출 시엔 gas 소비.

코드 예시

```
// view — 상태 읽기만
function balanceOf(address a)
    external view returns (uint256) {
        return balances[a];
    }

// pure — 입력만으로 계산
function toWei(uint256 eth)
    external pure returns (uint256) {
        return eth * 1e18;
    }

// payable — ETH 수신
function deposit()
    external payable {
        balances[msg.sender] += msg.value;
    }
```


함수 구조 — 가시성·상태 변경성 (3/3)

FUNCTION STRUCTURE — ERROR HANDLING (REQUIRE / REVERT / ASSERT)

require

- 입력 검증·조건 체크
- false → revert + 메시지
- 남은 gas 환불
- 외부 입력 검증 표준

```
require(amount > 0,  
  "Amount must be > 0");
```

revert + Custom Error

- 복잡한 조건 명시적 처리
- Custom Error → gas 절약
- 파라미터로 디버그 정보 포함
- 남은 gas 환불

```
error InsufficientBal(  
  uint256 have, uint256 need);  
  
if (bal < amt)  
  revert InsufficientBal(bal, amt);
```

assert

- 내부 불변 조건 검증
- false → Panic 에러
- **전체 gas 소비**
- 외부 입력 검증 절대 금지

```
// 버그 감지용  
assert(total == expected);  
  
// 실패 = 구현 버그
```

Custom Error 우선 (Solidity 0.8.4+) — string require보다 약 10% gas 절약. 함수 시그니처(4바이트)만 전송. OpenZeppelin v5 전환 완료.

assert 남용 금지 — 전체 gas를 소비하며 사용자 피해 직접 발생. 수학적 불변식(totalSupply 등) 검증에만 한정 사용.

05

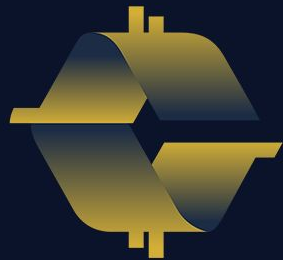
전역 컨텍스트 변수

msg · block · tx — 실행 환경에서 자동 제공되는 내장 변수

msg.sender

block.timestamp

tx.origin



COINCRAFT
ACADEMY

전역 컨텍스트 변수

GLOBAL CONTEXT VARIABLES — MSG / BLOCK / TX

변수	타입	내용
<code>msg.sender</code>	<code>address</code>	현재 함수를 호출한 주소 (직전 호출자)
<code>msg.value</code>	<code>uint256</code>	함께 보낸 ETH 양 (wei 단위)
<code>msg.data</code>	<code>bytes</code>	전체 calldata (함수 선택자 + 인자)
<code>block.timestamp</code>	<code>uint256</code>	현재 블록 UNIX 타임스탬프 (초)
<code>block.number</code>	<code>uint256</code>	현재 블록 번호
<code>tx.origin</code>	<code>address</code>	트랜잭션 최초 발신자 (EOA)
<code>gasleft()</code>	<code>uint256</code>	현재 시점 남은 gas 양

tx.origin 인증 금지 — 피싱 컨트랙트가 중간에 끼면 tx.origin은 피해자 주소. 항상 `msg.sender` 로 인증해야 한다.

block.timestamp 주의 — 마이너(검증자)가 ± 15 초 조작 가능. 복권·랜덤 시드 등 정밀 시간 로직에 사용 금지. VRF 사용 권장.

06

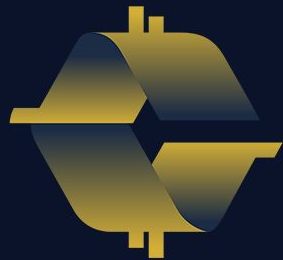
이벤트 트랜잭션 로그

indexed 필터링 · 오프체인 구독 · ERC 표준 이벤트

emit

indexed

The Graph



COINCRAFT
ACADEMY

이벤트 — 트랜잭션 로그

EVENTS — TRANSACTION LOG (OFFCHAIN COMMUNICATION)

선언과 발생

```
// 이벤트 선언 (최대 3개 indexed)
event Transfer(
    address indexed from,
    address indexed to,
    uint256 value // non-indexed
);

function transfer(address to, uint256 amt)
    external {
    balances[msg.sender] -= amt;
    balances[to]          += amt;
    emit Transfer(msg.sender, to, amt);
}
```

핵심 특성

- 트랜잭션 로그에 저장 — 상태변수보다 **~8배 저렴**
- `indexed` 파라미터 → 필터링 가능 (topic으로 저장)
- 컨트랙트 내부에서 이벤트 데이터 읽기 불가
- 오프체인(ethers.js, The Graph)에서 구독·필터링
- 최대 indexed 3개 제한

인덱서의 핵심 소스 — ERC-20 Transfer, ERC-721 Transfer 이벤트
가 Etherscan·The Graph의 데이터 기반. 반드시 emit해야 한다.

오프체인 필터 — `provider.on("Transfer", filter, callback)` 으
로 특정 주소 이체만 구독 가능.

07

modifier

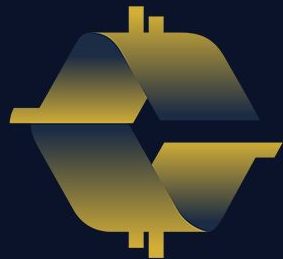
함수 전·후처리

실행 순서 · 위치 · 코드 인라이닝 · 상속 · constructor

onlyOwner

nonReentrant

whenNotPaused



COINCRAFT
ACADEMY

modifier — 함수 전·후처리 (1/3)

MODIFIER — REUSABLE PRE/POST CONDITION GUARD

기본 구조

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _; // ← 함수 본문 실행 위치
}

modifier whenNotPaused() {
    require(!paused, "Paused");
    _;
}

// 여러 modifier — 왼쪽부터 순서대로
function withdraw(uint256 amt)
    external
    onlyOwner      // ① 먼저
    whenNotPaused  // ② 다음
{
    // ③ 함수 본문
    payable(owner).transfer(amt);
}
```

실행 순서 (stack 구조)

① onlyOwner: require 실행 └─ ② whenNotPaused: require 실행 └─ ③ withdraw 본문 실행 ③ 종료 ② _; 이후 (없음) ① _; 이후 (없음)

중첩 순서 중요 — 왼쪽 modifier가 먼저 진입하고 나중에 빠져나온다. 보안 modifier는 항상 앞에 배치.

인자 있는 modifier — modifier onlyRole(bytes32 role) 형태로 파라미터 전달 가능.

modifier — 함수 전·후처리 (2/3)

MODIFIER — UNDERSCORE POSITION DETERMINES EXECUTION ORDER

⋮ 뒤 — 전처리 (가장 일반적)

```
modifier checkBefore() {  
    require(조건, "fail"); // ① 검증  
    ⋮;                      // ② 함수 본문  
}
```

// 접근 제어, 입력 검증에 사용

활용 예

onlyOwner, whenNotPaused, onlyRole 등
함수 실행 전 조건을 확인하는 모든 guard

⋮ 앞 — 후처리 (ReentrancyGuard)

```
modifier nonReentrant() {  
    require(!locked, "Reentrant");  
    locked = true; // ① lock  
    ⋮;             // ② 함수 본문  
    locked = false; // ③ unlock  
}
```

// 재진입(reentrancy) 공격 차단

Reentrancy 원리 — 외부 컨트랙트 호출 전 lock, 호출 후 unlock.
호출 중 재진입 시 locked=true → revert.

체크-이펙트-인터랙션 패턴 — modifier보다 코드 구조로 해결 권장. ① 검증 ② 상태 변경 ③ 외부 호출 순서 유지.

modifier — 함수 전·후처리 (3/3)

MODIFIER — INLINING / INHERITANCE / CONSTRUCTOR

코드 인라이닝

- modifier는 컴파일 시 함수 본문에 인라이닝 됨
- 런타임 함수 호출 오버헤드 없음
- 여러 modifier를 써도 gas는 로직 합계

modifier 10개를 써도 각각의 로직 gas만 소비. "modifier overhead"는 없다.

상속

```
contract Ownable {
    address public owner;
    modifier onlyOwner() {
        require(
            msg.sender == owner;
        );
    }
}

contract MyToken is Ownable {
    function mint()
        external onlyOwner { }
    // 상속된 modifier 사용
}
```

constructor

```
constructor(address _owner) {
    // 배포 시 1회만 실행
    owner = _owner;
    totalSupply = 1_000_000 ether;
    balances[_owner] = totalSupply;
    emit Transfer(
        address(0), _owner, totalSupply);
}
```

- 배포 시 1회 실행, 이후 호출 불가
- 상태변수 초기화·초기 토큰 발행에 사용

OpenZeppelin Ownable — 직접 구현하지 말고 OZ `Ownable.sol` 상속 권장. 검증된 구현체 재사용으로 버그 위험 제거.

08

실습

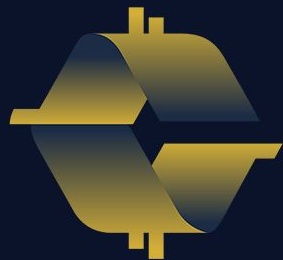
A · B · C

VotingToken · Storage 패킹 gas 비교 · Custom Error gas 비교

실습 20min

Hardhat

gasUsed 측정



COINCRAFT
ACADEMY

실습 A — VotingToken 핵심 시나리오

EXERCISE A — ERC-20 + VOTING WEIGHT

- 1 **VotingToken 배포** — totalSupply 1,000,000. constructor에서 전량 msg.sender에게 발행.
- 2 **5개 주소에 transfer** — 각 100 토큰. Transfer 이벤트 5회 발생 확인.
- 3 **approve → transferFrom** — 주소 A가 B에게 50 approve. B가 A로부터 40 transferFrom.
- 4 **이벤트 필터링** — Hardhat console에서 Transfer 이벤트 indexed 필터로 특정 주소 이체만 조회.
- 5 **불변식 검증** — 모든 주소 balanceOf 합산 = totalSupply. assert로 확인.

구현 체크리스트

- transfer: 잔액 부족 → Custom Error
- transferFrom: allowance 부족 → Custom Error
- emit Transfer / emit Approval 필수
- onlyOwner modifier로 mint 보호

검증 포인트 — totalSupply는 mint/burn 없는 한 항상 1,000,000.
transfer는 총량을 바꾸지 않는다.

Gas 참고 — transfer: ~51,000 gas / approve: ~46,000 gas /
transferFrom: ~58,000 gas (cold storage 기준)

실습 B — Storage 패킹 gas 비교

EXERCISE B — MEASURE SSTORE GAS DIFFERENCE BEFORE/AFTER PACKING

❌ Unpacked — 슬롯 4개

```
contract Unpacked {
  uint64 a; // slot 0
  uint256 b; // slot 1
  uint64 c; // slot 2
  uint128 d; // slot 3

  function setAll(
    uint64 _a, uint256 _b,
    uint64 _c, uint128 _d
  ) external {
    a=_a; b=_b; c=_c; d=_d;
  }
}
```

✅ Packed — 슬롯 2개

```
contract Packed {
  uint64 a; // slot 0 (8B)
  uint64 c; // slot 0 (8B)
  uint128 d; // slot 0 (16B) ← 괄참
  uint256 b; // slot 1

  function setAll(
    uint64 _a, uint256 _b,
    uint64 _c, uint128 _d
  ) external {
    a=_a; b=_b; c=_c; d=_d;
  }
}
```

측정 방법 — Hardhat: `const tx = await contract.setAll(...); console.log(tx.gasUsed)`. Packed가 약 40,000~60,000 gas 절약 예상 (cold SSTORE 22,100 × 슬롯 수 차이).

결론 — 슬롯 차이 2개 = SSTORE 절약 2회 = ~44,200 gas 절약. 자주 쓰이는 struct에서 패킹 효과 극대화.

실습 C — Custom Error vs string require gas 비교

EXERCISE C — MEASURE GAS SAVING FROM CUSTOM ERROR

❌ string require — 더 비쌈

```
function transferOld(  
    address to, uint256 amt  
) external {  
    require(  
        balances[msg.sender] >= amt,  
        "ERC20: transfer amount exceeds balance"  
    );  
    // 문자열 전체가 calldata에 포함  
    // ...  
}
```

✅ Custom Error — gas 절약

```
error InsufficientBalance(  
    address sender,  
    uint256 have,  
    uint256 need  
);  
  
function transfer(  
    address to, uint256 amt  
) external {  
    if (balances[msg.sender] < amt)  
        revert InsufficientBalance(  
            msg.sender, balances[msg.sender], amt);  
}
```

방식	revert 시 전송 데이터	gas 절약	디버그 정보
string require	문자열 전체 (ABI 인코딩)	기준	메시지 텍스트만
Custom Error	함수 시그니처 4바이트 + 파라미터	약 10~15% 절약	구조화된 파라미터 포함

OpenZeppelin v5 전환 완료 — string require → Custom Error 전환. 신규 프로젝트는 Custom Error를 기본으로 사용하라.